Sync/CC: Continuations and Aspects to Tame Callback Dependencies on JavaScript Handlers*

Paul Leger
Escuela de Ciencias Empresariales
Universidad Católica del Norte
Chile
pleger@ucn.cl

Hiroaki Fukuda
Shibaura Institute of Technology
Japan
hiroaki@shibaura-it.ac.jp

ABSTRACT

In JavaScript handlers, callbacks are widely used to develop rich responsive and interactive Web applications. However, the dependency among nested callbacks, known as callback hell, can make it difficult to understand and maintain them, which will eventually mix concerns. Unfortunately, current solutions for JavaScript do not fully address the aforementioned issue. This paper presents Sync/cc, a JavaScript library that uses continuations and aspects to allow developers to write handlers that need nested callbacks in a synchronous style, preventing callback dependencies. Unlike current solutions, Sync/cc is modular, customizable, and succinct because it does not require special and scattered keywords, code refactoring, or adding ad-hoc implementations like state machines. In practice, our proposal uses *a)* continuations to only suspend the current handler execution until the asynchronous operation is resolved, and *b)* aspects to apply continuations in a non-intrusive way.

CCS Concepts

•**Theory of computation** → *Algorithm design techniques;*
•**Information systems** → *Web applications;*

Keywords

Callback Dependencies, Aspect-Oriented Programming, Continuations, JavaScript, Sync/cc, AspectScript, Web applications

*An earlier version of this article appeared in the Workshop on Foundations of Aspect-Oriented Languages (FOAL 2016).

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

SAC 2017,April 03-07, 2017, Marrakech, Morocco

Copyright 2017 ACM 978-1-4503-4486-9/17/04…$15.00

http://dx.doi.org/10.1145/3019612.3019783

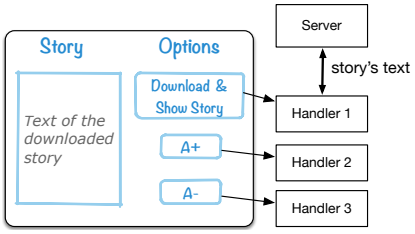

Figure 1: A JavaScript application to download and read stories on the Web.

1. INTRODUCTION

The software industry strongly focuses on Web applications, where many of them are interactive. JavaScript, a dynamic prototype-based language with higher-order functions, is one of the most used languages for developing these applications because all modern browsers support it. In the development of these applications, asynchronous programming has been adopted to hide latency of the network and improve responsiveness of the interactivity of a user. Figure 1 shows an interactive JavaScript application to read stories online. In this application, every option triggers a *handler*, and one of them (Handler 1) needs the network to retrieve data from a server.

In singled-thread languages like JavaScript, asynchronous programming developers replace an invocation that *blocks* the whole program execution with a non-blocking invocation that immediately returns. Whenever the associated process to the non-blocking invocation finishes, a piece of code known as *callback* is executed. We illustrate asynchronous programming and contrast with synchronous programming using a handler of a Web application that allows users to read stories available on a server when a button is pushed:

```
function storyButtonHandler_SYNC() {
  var URL = "http://storyServer.com?";
  var idStory = getElementById("idStory").value;

  var story = syncDownload(URL + "id=" + idStory);
  display(story);
}
```

Listing 1: A synchronous version of the story button handler.

```
1 function storyButtonHandler_ASYNC() {
2   var URL = "http://storyServer.com?";
3   var idStory = getElementById("idStory").value;
4
5   function callback(story) { display(story); }
6   asyncDownload(URL + "id=" + idStory,callback);
7 }
```

Listing 2: An asynchronous version of the story button handler.

In the synchronous programming version of the button handler (Listing 1), the execution of syncDownload blocks the handler work until this execution finishes (Line 6). Instead, in the asynchronous version (Listing 2), when the content of the selected story is downloaded from the server, the callback function is executed to display the story (Line 5). Unlike synchronous programming, the use of asynchronous programming splits a set of statements into an asynchronous operation (asyncDownload), and a callback that receives the operation result (story) and executes the rest of statements (display(story)). Although asynchronous programming is used in Listing 2, its implementation is still understandable because there is a single callback with no dependencies. For example, consider now an application to read books that contain chapters and images:

```
function bookButtonHandler() {
  var URL = "http://bookServer.com?";
  var idBook = getElementById("idBook").value;

  asyncDownload(URL + "id=" + idBook,function(book) {
    //display description about the book on the HTML
    book.chaptersURL.forEach(function(chapURL) {
      asyncDownload(chapURL,function(chapter) {
        //display content of the chapter on the HTML
        chapter.imagesURL.forEach(function(imgURL) {
          asyncDownload(imgURL,function(img) {
            //display images of the chapter on the HTML
          });
        });
      });
    });
  });
}
```

Listing 3: Nested asynchronous calls inside of a handler.

In bookButtonHandler of Listing 3, we can see nested callbacks because of their dependencies. This is because the two nested asynchronous calls can only be invoked when the response of the previous asynchronous operation is available. These callback dependencies, known as *callback hell* [13], make it difficult to understand and maintain pieces of code. In addition, these (coupled) callback dependencies eventually end up tangling concerns if the software is getting complex. For example, bookButtonHandler now tangles the download and display concerns.

For the Web application development, JavaScript developers can use libraries to address callback dependency issues [9, 10, 11, 14]. Most of them are fairly basic in that they at best, attempt to hide nested callbacks through the creation of artificial and dependent functions. Another proposal is Babel [10], which uses a code transformer to allow developers to write pieces of code in a synchronous manner using async/await keywords of C# [4] in JavaScript. However, the previous proposal scatters specialized keywords through the code and requires insertion of ad-hoc state machines for every async/await to control callback executions. In addition, the semantics of these keywords are fixed for developers.

This paper presents Sync/cc, a JavaScript library that uses continuations [6] and aspects of Aspect-Oriented Programming [7] to allow developers to write asynchronous pieces of code in a synchronous style, preventing callback dependencies, and therefore modularizing concerns. In Sync/cc, continuations are used to only suspend the handler execution that contains an asynchronous operation and not whole the program execution until the result of this operation is available; improving the responsiveness of the user. Aspects, through the intersection of asynchronous operations, of our proposal apply continuations in a non-intrusive way. Unlike existing proposals, Sync/cc:

a) fits completely within continuations and aspects, providing a modular solution. Hence, Sync/cc does not require specialized keywords that are scattered around methods that use asynchronous operations.

b) is customizable because its implementation utilizes only two user-defined aspects.

c) is succinct given that the Sync/cc implementation only requires 20 lines of indented code.

Sync/cc is currently implemented in Rhino [12], a JavaScript interpreter implemented in Java, and uses AspectScript [16], an aspect language for JavaScript. The next section reviews current proposals and potential programming language mechanisms that may be used in JavaScript. We then introduce Sync/cc through a new implementation of the function bookButtonHandler. Section 4 concludes and describes major challenges for our proposal.

Availability. Sync/cc with the two examples presented here (story and book handlers) are available online at http://pleger.cl/synccc.

2. RELATED WORK

Issues related to callback dependencies have already been identified, resulting in a number of proposals for the JavaScript language, classified into libraries and specialized keywords. We review these proposals and advances in programming language mechanisms even though these advances are not available in JavaScript yet.

2.1 Libraries

A number of JavaScript libraries is available on the Web [9, 11, 14], which takes advantage of first-class functions to mitigate callback hell issues. We illustrate this kind of solution with an implementation of bookButtonHandler using Async.Js [11]:

```
function bookButtonHandler_ASYNCJS() {
  var URL = //... as above
  var idBook = //... as above
  function downloadAndDisplayBook( continuation ) {
    asyncDownload(URL + "id=" + idBook,function(book) {
      //display description about the book on the HTML
      continuation(book);
  }); }
```

```
function downloadAndDisplayChapters(book, [continuation]) {
  book.chaptersURL.forEach(function(chapURL) {
    asyncDownload(chapURL, function(chap) {
      //display content of the chapter on the HTML
      [continuation(chapter)];
});});}); }

function downloadAndDisplayImages(chapter, [continuation]) {
  chapter.imagesURL.forEach(function(imgURL) {
    asyncDownload(imgURL, function(img) {
      //display images of the chapter on the HTML
      // [No more continuations]
});});}); }
Async.waterfall([downloadBookAndDisplay,
                 downloadAndDisplayChapters,
                 downloadAndDisplayImages]);
}
```

Listing 4: The book handler implementation using Async.Js.

In Listing 4, the Async.waterfall method allows developers to use a *continuation-passing style* pattern [2] to "visually" hide callback dependencies. However, similarly to the original implementation of this handler (Listing 3), developers must be aware of the dependency chain of callbacks. In addition, this library enforces a non-common programming style for developers, affecting understandability [13].

2.2 Specialized Keywords

Babel [10] is a JavaScript code transformer that allows developers to use non-native features like async/await of C# [4][1]. In async/await, the tagged async functions can use the await keyword, which suspends the current executing function and yields control to the caller of the async method until the awaited asynchronous operation is resolved.

Listing 5 shows an implementation of the book button handler using async/await of Babel. This implementation shows that the use of async/await permits avoiding callback dependencies and modularizing concerns; for example, the display concern is not inside callbacks. In Line 5, the handler execution is suspended until the value of book is available. In all calls of asyncDownload, the last parameter is now a function with an empty body because this callback is no longer necessary. This is because the handler execution cannot continue until the value is available or an exception is triggered.

```
1  async function bookButtonHandler_BABEL() {
2    var URL = //... as above
3    var idBook = //... as above
4
5    var book = await asyncDownload(URL + "id=" + idBook,
6                function(book){});
7    var chapters = book.chaptersURL.map(async function(chapURL) {
8          return await asyncDownload(chapURL, function(chap){});});
9    var images = chapters.map(function(chapter) {
10     return chapter.imagesURL.map(async function(imgURL) {
11       return await asyncDownload(imgURL, function(img){});
12   });});
13
14   //display book description, chapters, and images on the HTML
15 }
```

Listing 5: The book button handler implementation using async/await of Babel.

[1]The async/await keywords with their associated behavior may be supported in the next JavaScript specification (ECMAScript 7): https://tc39.github.io/ecmascript-asyncawait.

Unfortunately, the async/await keywords are scattered around the function body and the callers of this function. For example, these two keywords appear six times in the implementation of bookButtonHandler_BABEL. In addition, async/await are fixed, meaning that developers cannot customize these keywords for specific scenarios and end up "coding around" these keywords. For example, an asynchronous invocation to multiple servers may require an await-for-the-first, which only waits for the first response. Finally, Babel transforms every function that contains async/await into nested and specialized state machines (like C# solution), making the debugging task for developers difficult.

2.3 Programming Language Mechanisms

Programming languages provide several mechanisms that may contribute to the solution of asynchronous issues. We will briefly review two of them: *lazy evaluation* [8] and *Future* [3].

Lazy evaluation. A programming language uses a strategy which determines when an expression is evaluated, particularly arguments of a function call. We briefly explain *lazy evaluation* that may address asynchronous issues by comparing *eager evaluation* that many programming languages adopt including JavaScript.

```
1  function addOne(num) {
2    return num + 1;
3  }
4  function display(v) {
5    printOnTheScreen(v);
6  }
7  display(addOne(10)); // display 11
```

Listing 6: Piece of code used to compare evaluation strategies.

Listing 6 defines two functions: addOne which returns its arguments increased by one, and display just displays the argument. In Line 7, display is invoked with an argument, value returned from addOne with 10, displaying 11.

In the *eager evaluation* strategy, the argument is agressively evaluated, meaning that the invocation of addOne is done before invoking display. As a result, the number 11 is directly passed to addOne. In this strategy, the next computation (*e.g.,* display) is always blocked until the current computation (*e.g.,* addOne) finishes.

In contrast, in the *lazy evaluation* strategy, the evaluation of arguments is deferred when the value is truly required. For example, the invocation of addOne is not done until the program execution reaches Line 5 where printOnTheScreen requires the value of v. In this strategy, the next computation (*e.g.,* display) may not be blocked even though the current computation (*e.g.,* addOne) may have not finished.

```
1  function downloadImage(url) {
2    // download image
3  }
4
5  function init() {
6    var image = downloadImage(url);
7    Window.onClick = function(e) {
8      // click handler on Window
9    }
10   display(image);
11 }
```

Listing 7: Applying lazy evaluation to a Web application.

Lazy evaluation may help addressing blocking issues as follows. Listing 7 defines two functions: downloadImage downloads an image, init initializes an application – we assume that downloadImage takes certain time for downloading. The init function first *a*) downloads an image (Line 6), then *b*) adds a handler when there is a "click" on the window (from Line 7 to 9), and init finally *c*) shows the downloaded image on the screen (Line 10). Given that JavaScript adopts an eager evaluation, the init execution is blocked until downloadImage finishes, resulting in the click handler not being able to be added. Therefore, applying lazy evaluation enables adding the click handler because image is truly required in Line 10, meaning that the execution of downloadImage is deferred. Although the lazy evaluation strategy seems to solve the blocking problem, this is because we assume that an asynchronous operation (*e.g.,* downloadImage) takes time to finish and this operation does not start until the data is really needed (*e.g.,* display). In other words, we would like to start downloadImage first while the click handler is being added. Instead lazy evaluation never starts downloadImage until the program execution reaches Line 10.

Future. A future value (simply future) refers to a value which is bound to a result of the execution of a separate thread. The concept was born to synchronize program executions in some concurrent programming languages. One of the difficult points in cuncurrent programming is the synchronization with a returned value where a parent thread delegates a task to a child thread, then the parent thread waits for the returned value from a child thread at a certain point. The future makes this operation easier as shown in Listing 8.

```
1   function downloadImage(url) {
2     // download image
3   }
4
5   function init() {
6     var future = Task.start(downloadImage,url);
7     Window.onClick = function(e) {
8       // click handler on Window
9     }
10    display(future.get()); //block until future is available
11  }
```

Listing 8: Applying *future* to a Web application.

Listing 8 rewrites Listing 7 using future. We assume that Task is an abstraction to represent a thread. In Line 6, a future value is immediately returned while Task starts executing downloadImage, thereby, the piece of code that adds click handler is never blocked. Then, future.get() gets an image which is downloaded using a thread, then display renders the image on the screen. Note that if the downloadImage is not finished before invoking display, the main thread will be blocked until its completion. The use of future allows developers to enable to solve asynchronous operation we have disscussed so far, however, this solution essentially contains the drawbacks of async/await. This is because both solutions are abstractions of synchronizing a concurrent programming model.

3. Sync/cc

Showing a new version of bookButtonHandler, this section presents Sync/cc. Similar to Babel, Sync/cc does not nest callbacks, nor tangle concerns. Unlike Babel, Sync/cc does not require:

a) scattering (and tangling) keywords like async and await at many places in the code, bringing modular benefits for Sync/cc developers.

b) "coding around" implementations to satisfy unforeseen needs because the only two aspects used by Sync/cc are customizable.

c) transforming the source code in order to insert a new implementation of a specialized state machine every time async/await is used. Sync/cc only requires defining aspects once. Note that we expect JavaScript may support general-purpose programming features like aspects and continuations in the future.

Listing 9 presents the bookButtonHandler implementation with Sync/cc. Every time that asyncDownload is called, Sync/cc returns the control to the application until the response of this operation is available. Our proposal then resumes the handler execution from the left hand assignment of each asyncDownload call (*e.g.,* var book = ...).

```
function bookButtonHandler_SYNC_CC() {
  var URL = //... as above
  var idBook = //... as above

  var book = asyncDownload(URL+"id="+idBook,function(book){});

  var chapters = book.chaptersURL.map(function(chapURL) {
      return asyncDownload(chapURL,function(chap){});
  });

  var images = chapters.map(function(chapter) {
    return chapter.imagesURL.map(function(imgURL) {
      return asyncDownload(imgURL,function(img){});
  });});

  //display book description,chapters, and images on the HTML
}
```

Listing 9: The book handler implementation using Sync/cc.

3.1 Use of Continuations in Sync/cc

This section briefly introduces continuations and then explains how Sync/cc uses them. Programming languages like Scheme provide continuations [6] which capture and store the current program control state as a first-class value. If this value is a function, it can be called and the current continuation will be replaced with the stored continuation. Developers in Rhino [12] captures the current continuation with new Continuation(). We illustrate continuations in Rhino through a piece of code that captures the execution of a function that adds two numbers (Listing 10):

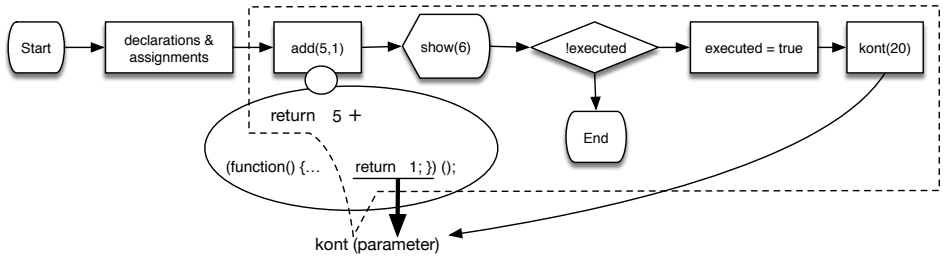

Figure 2: The flowchart of Listing 10, which uses the continuation kont. The invocation of kont uses a parameter to replace the return value of the anonymous function inside of add (*i.e.,* "return 1").

```
1 var kont;
2 function add(x,y) {
3   return x +
4          (function() {kont = new Continuation(); return y;})();
5 }
6 var executed = false;  //to prevent an infinite recursion
7 show(add(5,1));        //show 6
8 if (!executed) {executed = true; kont(20);} // show 25
```

Listing 10: Use of continuations in Rhino.

Figure 2 shows a flowchart of the piece of code above. In Line 4, the piece of code captures and stores the continuation in kont before adding y. This capture takes place in Line 8 when add is called. The result of add is passed to show, then the number 6 is displayed. Line 9 executes the continuation bound to kont with 20 as parameter, which replaces the return of the anonymous function of Line 4 (*i.e.,* return 1 → return 20). As a consequence, 25 is displayed (25 = (x = 5) + (y = 20)). Note that the flag executed is used to execute the continuation (kont(20)) only once and not infinitely.

If Rhino developers need to stop a handler execution, a continuation on the top-level script should be created. For example, Listing 11:

```
var stop = new Continuation();
function showMessageHandler() {
  show("This message will be shown");
  stop();
  show("This message will never be shown");
}
```

Listing 11: Stopping a JavaScript script with continuations in Rhino.

Sync/cc uses two continuations: the first one stops a handler execution just after an asynchronous operation invocation, and the second one resumes the handler execution when the asynchronous operation response is available.

3.2 Use of Aspects in Sync/cc

This section briefly explains Aspect-Oriented Programming [7] using AspectScript [16]. We then explain how this aspect language is used in Sync/cc.

In the pointcut-advice model of aspect-oriented programming, as embodied in *e.g.,* AspectScript, crosscutting behavior is defined by means of *pointcuts* and *advices*. Execution points at which an advice may be executed are called *(dynamic) join points*. A pointcut matches a set of join points, and an advice that is the action to be taken *before*, *around*, or *after* the matched join point. An around advice can invoke the computation of the join point matched, known as the *proceed* invocation.

In AspectScript, an aspect is a JavaScript object with three properties: a pointcut, an advice, and an advice kind. The two first properties are functions that receive a join point object as parameter. The advice kind property is a constant that indicates when the advice is executed (*i.e.,* before, around, or after). For example, the aspect of Listing 12 shows a message after an execution of bookButtonHandler:

```
var notificationOfBookHandler = {
  pointcut: function(jp) {
   //if jp is an execution of the bookButtonHandler function
   return jp.isExec() && jp.fun == bookButtonHandler
  },
  advice: function(jp) {
   //function executed when jp is matched
   show("bookButtonHandler was executed");
  },
  kind: AFTER
};

//deployment of the aspect
AspectScript.deploy(notificationOfBookHandler);
```

Listing 12: An aspect that shows a message when a handler is executed.

Listing 13 shows the complete implementation of Sync/cc, which uses two aspects. The first aspect executes its advice around an asynchronous operation call. The second aspect triggers its advice after the execution of the associated callback to the asynchronous operation; in the book handler example, Listing 9, the callback is the second and last parameter of asyncDownload.

```
var kont, stop = new Continuation();
var asyncOperation = {
    pointcut: function(jp) {
      return jp.isCall() && jp.fun == asyncDownload;},
    advice: function (jp) {
      ① kont = new Continuation();
      var response = ② jp.proceed(); //executing asyncDownload
      ③ stop();
      return response;
    },
    kind: AROUND
};
```

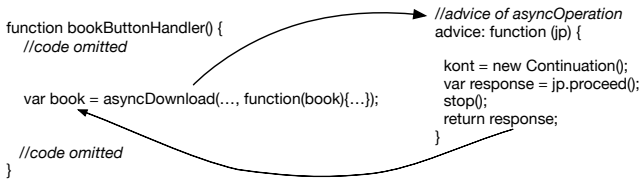

Figure 3: Behavior of the asyncOperation advice to return a value to the assignment of an asynchronous operation.

```
var asyncCallback = {
    pointcut: function(jp) {
      return jp.isExec() && jp.fun == callback
    },
    advice: function (jp) {
      var response = jp.args[0]; //1st arg to callback
      kont(response); //resuming the asyncOperation advice
    },
    kind: AFTER
};
```

Listing 13: Implementation of Sync/cc.

The asyncOperation aspect matches the call of asyncDownload and its around advice carries out three tasks. The task ① creates a continuation (kont) that represents the execution of this advice and the rest of the handler execution. The task ② executes the original computation. Finally, the advice stops the handler execution using the stop continuation (task ③). The asyncCallback aspect matches the callback execution and, its advice resumes the kont continuation with the response of the asynchronous operation. As a consequence, the execution of the asyncOperation advice, which replaces the asyncDownload execution, is resumed to return the already available response of the operation (Figure 3). Thereby, the implementation of bookButtonHandler with Sync/cc (Listing 9) can now use the Array.map method instead of Array.forEach, which is used in the original implementation of this handler (Listing 3) because it only iterates the array without returning a new array as a result.

4. CONCLUSIONS

Because there is a strong push to build complex, interactive, and highly responsive JavaScript applications, appropriate abstractions for asynchronous programming is crucial. In this kind of programming, callbacks are widely used nowadays. Unfortunately, callback dependencies (*a.k.a.*callback hell) are likely to appear and make it difficult to eventually understand and maintain pieces of code that crosscut concerns. Current proposals do not fully address callback dependencies. Using general concepts in programming languages, this paper proposes Sync/cc: a JavaScript library to prevent callback dependencies using continuations and aspects. Continuations prevent the need to execute statements in callback bodies and aspects apply continuations without the need to modify the code. Regarding future work, the major challenges for Sync/cc are:

Concrete implementation. Because widely used browsers like Google Chrome do not yet natively support continuations, Sync/cc is currently implemented in Rhino. To have a Sync/cc implementation that works on modern browsers we plan to use tools like CPS-JS [15] to *partially* transform JavaScript scripts to continuation-passing style [2] in order to implement continuations.

Performance. For a real use of Sync/cc, its performance is crucial. Although this paper does not discuss the performance topic, the use of continuations negatively affects the Sync/cc performance. Fortunately, there are studies like the one found in [1] that address how to compile with continuations. We plan to apply techniques of compilation to optimize Sync/cc. Compiling JavaScript scripts is already used in modern browsers. For example, the JavaScript engine V8 [5] of Google Chrome compiles scripts to create *hidden classes* in order to improve performance.

Acknowledgments. We are grateful to Dominique Leger for her help in clarifying our initial intuition. This work was partially supported by JSPS KAKENHI Grant Numbers 2633089.

5. REFERENCES

[1] A. W. Appel. *Compiling with Continuations*. Cambridge University Press, 1992.

[2] A. W. Appel and T. Jim. Continuation-passing, closure-passing style. In *Proceedings of the 16th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL 89)*, pages 293–302, New York, NY, USA, 1989. ACM.

[3] H. C. Baker, Jr. and C. Hewitt. The incremental garbage collection of processes. *SIGPLAN Not.*, 12(8):55–59, Aug. 1977.

[4] N. Benton, L. Cardelli, and C. Fournet. Modern concurrency abstractions for C♯. *ACM Transactions on Programming Languages and Systems*, 26(5):769–804, Sept. 2004.

[5] G. Chrome. Chrome v8: Open source JavaScript engine, 2012. https://developers.google.com/v8/.

[6] D. P. Friedman and M. Wand. Reification: Reflection without metaphysics. In *Proceedings of the Annual ACM Symposium on Lisp and Functional Programming*, pages 348–355, Aug. 1984.

[7] G. Kiczales, J. Irwin, J. Lamping, J. Loingtier, C. Lopes, C. Maeda, and A. Mendhekar. Aspect oriented programming. In *Special Issues in Object-Oriented Programming*. Max Muehlhaeuser (general editor) et al., 1996.

[8] J. Launchbury. A natural semantics for lazy evaluation. In *Proceedings of the 20th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, pages 144–154, Charleston, South Carolina, USA, 1993. ACM.

[9] F. Lindesay. Promise: A library for promises in JavaScript, Dec. 2012. https://www.promisejs.org/.

[10] S. McKenzie. Babel: A compiler for writing ES6 and ES7 generation JavaScript, Jan. 2014.

[11] C. McMahon. Async.Js: A library for working with asynchronous JavaScript, June 2010. https://github.com/caolan/async.

[12] Mozilla. Rhino: An open-source implementation of JavaScript written entirely in Java, 1997. https://developer.mozilla.org/en-US/docs/Mozilla/Projects/Rhino.

[13] M. Ogden. Callback hell, 2016.

[14] A. ONeal. FuturesJS: Asynchronous library in JavaScript using futures, 2007. https://github.com/FuturesJS.

[15] C. Perkins. CPS-JS: A JavaScript library for converting synchronous code into the continuation-passing, 2011. https://github.com/cperkins/js-cps.

[16] R. Toledo, P. Leger, and É. Tanter. AspectScript: Expressive aspects for the Web. In *Proceedings of the 9th ACM International Conference on Aspect-Oriented Software Development (AOSD 2010)*, pages 13–24, Rennes and Saint Malo, France, Mar. 2010. ACM Press.